SAVEETHA SCHOOL OF ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

C PROGRAMMING ASSIGNMENT

Student Submission Template

Student Name	P H Devesh
Register Number	1911260236
Course Code & Name	CSA0209 · C Programming
Slot	B
Assignment / Task Title	Design, Implement and Validate a Reliable Billing Program
GitHub Repository URL	Repository: CSA02-1912260571-Assignment1
Submission Date	

Instructions to Students

1. Complete every section for each assigned task.
2. Write your own explanation, algorithm/pseudocode, and program.
3. Run the program and insert a clear screenshot of the output.
4. Include test cases and a short analysis where applicable.
5. If code is maintained on GitHub, provide the correct repository link.

Executive Summary

This report designs a reliable C billing program for a rural electricity co-operative. The original tariff is a fixed charge of Rs 50, the first 100 units are free, units 101–200 cost Rs 2.25 each, and units above 200 cost Rs 4.50 each. Money is represented as integer paise to avoid floating-point rounding errors. A 16-bit unsigned meter register is handled with unsigned arithmetic, and input validation prevents invalid readings and accidental negative bills.

The design is modified for smart meters that send watt-hour readings every 15 minutes, peak-hour consumption that costs 20% more, and tariff updates loaded at runtime from a configuration file. A production system should also persist validated readings, handle communication failures, audit tariff changes, and use appropriate wider integer types.

TASK 1 – Understand the Problem and Identify the Data (4 marks)

Inputs, outputs, constants and rules

Item	Data / example	Justification
Household number	String or unsigned integer ID	Identifies the account; a string also supports leading zeroes and mixed IDs.
Previous meter reading	unsigned short (16-bit register)	Valid register range is 0–65535; the original task specifies a 16-bit counter.
Current meter reading	unsigned short	Same register range and representation as the previous reading.
Units consumed	unsigned int / uint32_t	Difference may cross a rollover; a wider result is safer than the register type.
Tariff rates	Integer paise constants	Rates are 0, 225 and 450 paise per unit; revisions should be centralised.
Fixed charge	long long, in paise	5000 paise; avoids fractional rupee representation.
Energy charge and total	long long, in paise	Can be large at district scale; use checked arithmetic in production.
Category / tariff plan	enum or validated string	Allows category-specific rules without unexplained numeric codes.
Outputs	Units, slab-wise energy charge, fixed charge, total in Rs.xx	A bill should be non-negative and the total must equal its components.

Rules and assumptions

- Register rollover is one wrap from 65535 to 0; the register has modulus 65536.
- Consumption is current − previous when current is not smaller; otherwise consumption is (65536 − previous) + current.
- A negative bill is never valid. The fixed charge is applied even when consumption is zero.
- Tariff slabs are progressive: first 100 units at 0 paise, next 100 at 225 paise, remaining units at 450 paise.

- Rates and limits are illustrative assignment values; taxes, subsidies, arrears and meter faults are outside the original model.
- All calculations remain in integer paise until display. Inputs are validated before computation.

Alternative: readings stored as text

A reading such as "01530" stored as a string needs O(d) time to validate and convert, where d is the number of digits, and O(d) bytes of storage. An integer reading uses constant storage for a fixed-width type and arithmetic is O(1). Text preserves leading zeroes and can be useful for raw device logs, but conversion must reject non-digits and overflow. For billing calculations, parse once into a validated integer.

TASK 2 – Choose the Data Types (4 marks)

Type / item	Typical size on common systems	Range / use
char	1 byte	Character; at least 8 bits in common systems.
unsigned short	Often 2 bytes	0–65535 when 16-bit; verify with sizeof and limits.h.
int	Often 4 bytes	At least −32767 to 32767 by C standard; commonly about ±2.1 billion.
long	4 or 8 bytes	Platform dependent; not a portable assumption for large totals.
long long	At least 8 bytes	Suitable for large paise totals with checked overflow.
float	Often 4 bytes	Approximate decimal values; not appropriate for money.
double	Often 8 bytes	Higher precision but still binary floating point, not exact decimal paise.
uint64_t / int64_t	8 bytes if provided	Fixed-width option from stdint.h; good for large counters and totals.

Money should not be stored as float rupees because decimal fractions such as 2.25 are not always represented exactly in binary floating point. Store money as an integer number of paise instead. The second rollover example is 256 because unsigned 16-bit arithmetic wraps modulo 65536: (65536 − 65400) + 120 = 256. It is not negative because unsigned arithmetic does not represent negative values.

Floating-point accumulation can disagree with integer paise. For example, repeated binary approximations of 0.1 may accumulate to a value slightly below or above the mathematical result; formatting then rounds the result. Integer paise stores each exact bill amount. Addition is O(1) for either representation, and storage per value is O(1); the integer approach is exact within its range.

Complete Code – TODO 1

```
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;
unsigned int consumption(meter_t previous, meter_t current)
{
 /* Unsigned 16-bit subtraction wraps modulo 65536. */
 return (unsigned int)(meter_t)(current - previous);
```

```
}
int main(void)
{
 printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
 sizeof(meter_t), (unsigned int)USHRT_MAX);
 printf("Consumption: %u units\n", consumption(1200, 1530));
 printf("Consumption: %u units\n", consumption(65400, 120));
 return 0;
}
```

Output

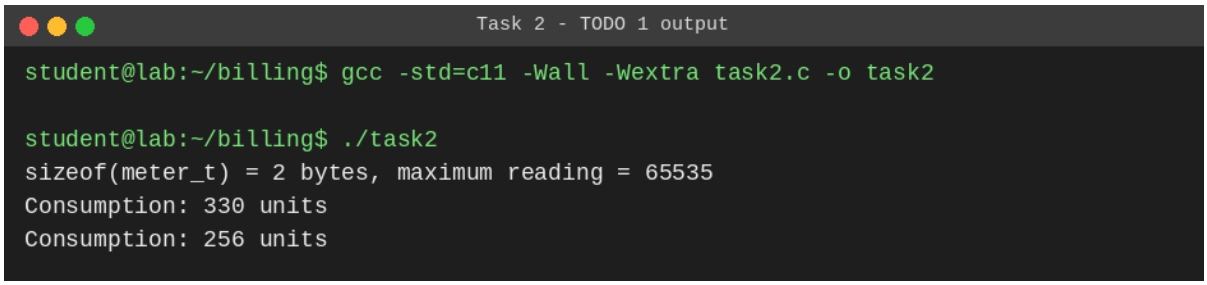


Figure 1: Output of Task 2 (TODO 1) – sizeof and rollover consumption

Expected and observed output match: sizeof(meter_t) = 2 bytes, maximum reading = 65535; Consumption: 330 units; Consumption: 256 units. C guarantees unsigned arithmetic modulo the range of the unsigned type, but the exact size of unsigned short is implementation-dependent, so the size must be taken from the target compiler (here it is 2 bytes).

TASK 3 – Define the Constants (3 marks)

```
#include <stdio.h>
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL
long long compute_bill(int units)
{
 long long paise = FIXED_CHARGE_PAISE;
 int first = units < SLAB1_UNITS ? units : SLAB1_UNITS;
 int second = units > SLAB1_UNITS
 ? (units < SLAB2_UNITS ? units - SLAB1_UNITS
 : SLAB2_UNITS - SLAB1_UNITS) : 0;
 int third = units > SLAB2_UNITS ? units - SLAB2_UNITS : 0;
 paise += (long long)first * RATE1_PAISE;
 paise += (long long)second * RATE2_PAISE;
 paise += (long long)third * RATE3_PAISE;
 return paise;
}
int main(void)
{
 int tests[] = {0, 100, 150, 200, 350};
 int n = (int)(sizeof tests / sizeof tests[0]);
 for (int i = 0; i < n; ++i) {
 long long p = compute_bill(tests[i]);
 printf("%d units -> Rs %lld.%02lld\n",
 tests[i], p / 100, p % 100);
 }
```

```
 return 0;
}
```

Output

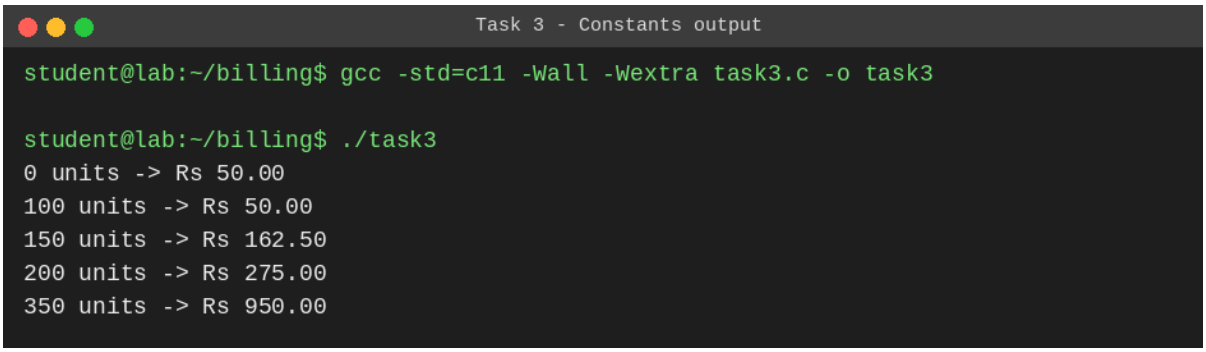


Figure 2: Output of Task 3 – bills for 0, 100, 150, 200 and 350 units

Changing RATE2_PAISE from 225 to 230 changes every bill with units in the second slab consistently because the rate is defined once. #define performs textual substitution before compilation and has no type checking or storage. A const variable has a type and scope and is easier to inspect in a debugger; an enum is suitable for named integral choices such as tariff categories. Use uppercase names for macros and descriptive names for const variables.

TASK 4 – Use Macros Safely (4 marks)

```
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
```

Preprocessor expansion can be inspected with gcc -E source.c. Parentheses are essential: SQUARE(x) written as x*x makes SQUARE(a+1) expand to a+1*a+1, which follows operator precedence and is not $(a+1)^2$. The safe macro expands to ((a+1) * (a+1)). MAX(i++, j) may evaluate i++ more than once because the selected argument can be evaluated again; avoid side effects in macro arguments.

Demonstration program

```
#include <stdio.h>
#define SQUARE(x) ((x) * (x))
#define BAD_SQUARE(x) x*x
#define MAX(a, b) ((a) > (b) ? (a) : (b))
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
int main(void)
{
 int a = 2;
 printf("BAD_SQUARE(a+1) = %d\n", BAD_SQUARE(a+1));
 printf("SQUARE(a+1)     = %d\n", SQUARE(a+1));
 printf("MAX(3, 7)       = %d\n", MAX(3, 7));
 printf("RUPEES_TO_PAISE(50) = %lld\n", RUPEES_TO_PAISE(50));
 return 0;
}
```

Output (including gcc -E expansion)

Task 4 - Macros output

```
student@lab:~/billing$ gcc -std=c11 -Wall -Wextra task4.c -o task4

student@lab:~/billing$ ./task4
BAD_SQUARE(a+1) = 5
SQUARE(a+1)     = 9
MAX(3, 7)       = 7
RUPEES_TO_PAISE(50) = 5000
student@lab:~/billing$ gcc -E task4.c | tail -8
{
 int a = 2;
 printf("BAD_SQUARE(a+1) = %d\n", a+1*a+1);
 printf("SQUARE(a+1)     = %d\n", ((a+1) * (a+1)));
 printf("MAX(3, 7)       = %d\n", ((3) > (7) ? (3) : (7)));
 printf("RUPEES_TO_PAISE(50) = %lld\n", ((long long)(50) * 100LL));
 return 0;
}
```

Figure 3: Macro demonstration and preprocessor expansion (gcc -E)

With a = 2, the unsafe version gives a+1*a+1 = 5, while the parenthesised SQUARE gives 9. A static inline function is often safer because each argument is evaluated once and types are checked. A function call has a small call/return cost in non-inlined builds, while a macro is expanded at each use and may increase code size. With optimisation, an inline function may be inlined, so the practical cost is often similar. Both approaches are O(1) for these arithmetic operations.

TASK 5 – Header Files and Conditional Compilation (3 marks)

```
/* billing.h */
#ifndef BILLING_H
#define BILLING_H
#define FIXED_CHARGE_PAISE 5000LL
long long compute_bill(int units);
#endif

/* Debug statement in a .c file */
#ifdef DEBUG
#define DEBUG_PRINT(...) printf(__VA_ARGS__)
#else
#define DEBUG_PRINT(...) ((void)0)
#endif
```

A C program normally contains header includes, macro definitions, declarations/prototypes, main(), and helper functions. Include guards prevent a header from being processed repeatedly. #include <file> searches standard include locations; #include "billing.h" searches the current/source include path first. Compile with gcc -DDEBUG billing.c main.c -o billing to enable debugging output. With #ifdef, debug statements are removed when DEBUG is not defined; a runtime if (debug) keeps the test and debug code in the executable, adding a small per-check runtime cost and code size.

TASK 6 – Read and Print Data Correctly (4 marks)

Use format specifiers that match the exact type: %d for int, %u for unsigned int, %ld for long, %lld for long long, %f for double in printf, %lf for double in scanf, %c for char and %s for a character array. A

mismatch such as %d for a long is undefined behaviour. Always check scanf's return value; it should equal the number of expected conversions.

```
int units;
if (scanf("%d", &units) != 1 || units < 0) {
    printf("Invalid input\n");
    return 1;
}
long long paise = 16250;
printf("Bill = Rs %lld.%02lld\n", paise / 100, paise % 100);
```

If input is read with fgets() and then parsed using strtol(), the program can reject trailing junk, detect range errors and recover by discarding a whole invalid line. Processing a line of length L takes O(L) time and O(L) buffer space. scanf is concise but line recovery is less straightforward; fgets plus strtol is preferable for robust interactive input.

TASK 7 – Control Type Conversion and Precision (4 marks)

- Integer division 5 / 2 produces 2, not 2.5. Use 5.0 / 2 or integer paise arithmetic as appropriate.
- Converting a large long long to int can lose information or produce an implementation-defined result if out of range; do not narrow totals.
- Mixing signed and unsigned values can convert a negative signed value to a large unsigned value; validate inputs before arithmetic.
- Rounding fractional charges should follow one documented rule. For a 20% peak surcharge, calculate in paise and round once per billing interval or bill according to policy.

Expression	Unexpected result	Reason / correction
printf("%d", 5/2)	2	Integer division truncates; use 5.0/2 for a fractional result.
SQUARE(1+2) with x*x	5 (1+2*1+2)	Precedence issue; define SQUARE(x) ((x)*(x)) to get 9.
float total = 0.1f repeated 1000 times	May not equal exactly 100.0	Binary approximation; use integer paise for money.
unsigned int u = 0; u - 1	Large positive value	Unsigned arithmetic wraps; validate before subtraction.

If all calculations use float and are rounded only at the end, rounding error can accumulate with the number of bills and the order of addition can affect the result. Integer paise addition is exact unless the integer type overflows. Both are O(1) per addition and O(1) space per total, but integer paise is recommended for deterministic bills.

TASK 8 – Trace the Compilation Process (4 marks)

Command	Output / purpose
gcc -E billing.c -o billing.i	Preprocessed C: headers expanded and macros replaced.
gcc -S billing.c -o billing.s	Assembly source generated.
gcc -c billing.c -o billing.o	Object file generated; not yet linked.

gcc billing.o main.o -o billing	Link object files and libraries into executable.
gcc -Wall -Wextra -Wconversion billing.c -o billing	Build with useful warnings; fix warnings rather than ignoring them.

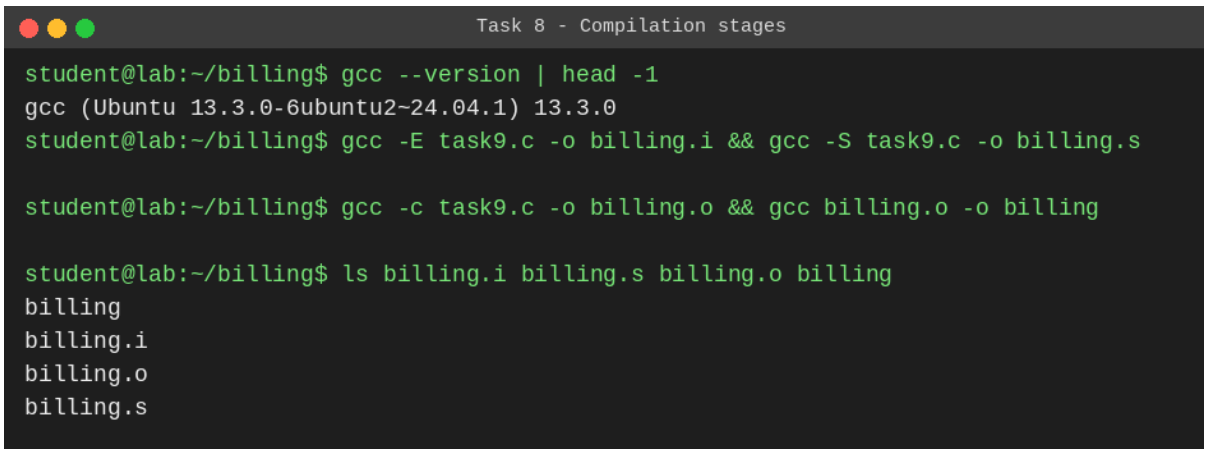


Figure 4: Compilation stages (.i, .s, .o and executable) produced by GCC

A compile-time error is detected during compilation (for example, a missing semicolon). A link-time error occurs when a referenced function has no definition (for example, an undefined reference to compute_bill). A run-time error occurs during execution (for example, invalid input or division by zero). For a multi-file project, only changed source files need recompilation, then the linker rebuilds the executable. This reduces build time for large projects; each object file uses additional disk space.

TASK 9 – Implement and Test the Billing Program (4 marks)

```
#include <stdio.h>
#include <limits.h>
#define SLAB1_UNITS 100U
#define SLAB2_UNITS 200U
#define RATE2_PAISE 225LL
#define RATE3_PAISE 450LL
#define FIXED_PAISE 5000LL
#define METER_MODULUS 65536U
unsigned int get_units(unsigned int previous, unsigned int current)
{
 return (current - previous) & 0xFFFFU;
}
long long bill_paise(unsigned int units)
{
 unsigned int slab2 = units > SLAB1_UNITS
 ? (units < SLAB2_UNITS ? units - SLAB1_UNITS : 100U) : 0U;
 unsigned int slab3 = units > SLAB2_UNITS ? units - SLAB2_UNITS : 0U;
 return FIXED_PAISE + (long long)slab2 * RATE2_PAISE
 + (long long)slab3 * RATE3_PAISE;
}
int main(void)
{
 unsigned int previous, current, units;
 printf("Enter previous and current 16-bit readings: ");
 if (scanf("%u %u", &previous, &current) != 2 ||
 previous > 65535U || current > 65535U) {
 printf("Invalid input. Readings must be 0..65535.\n");
 return 1;
```

```
 }
 units = get_units(previous, current);
 {
 long long p = bill_paise(units);
 printf("Units consumed: %u\n", units);
 printf("Fixed charge: Rs 50.00\n");
 printf("Total bill: Rs %lld.%02lld\n", p / 100, p % 100);
 }
 return 0;
}
```

Compile with: gcc -std=c11 -Wall -Wextra billing.c -o billing (no warnings were produced). For Dev-C++ on Windows, create a C source file, paste the code, save as billing.c, and use Compile & Run. This code assumes at most one rollover between readings; smart-meter systems should also validate timestamps and plausibility.

Output screenshots

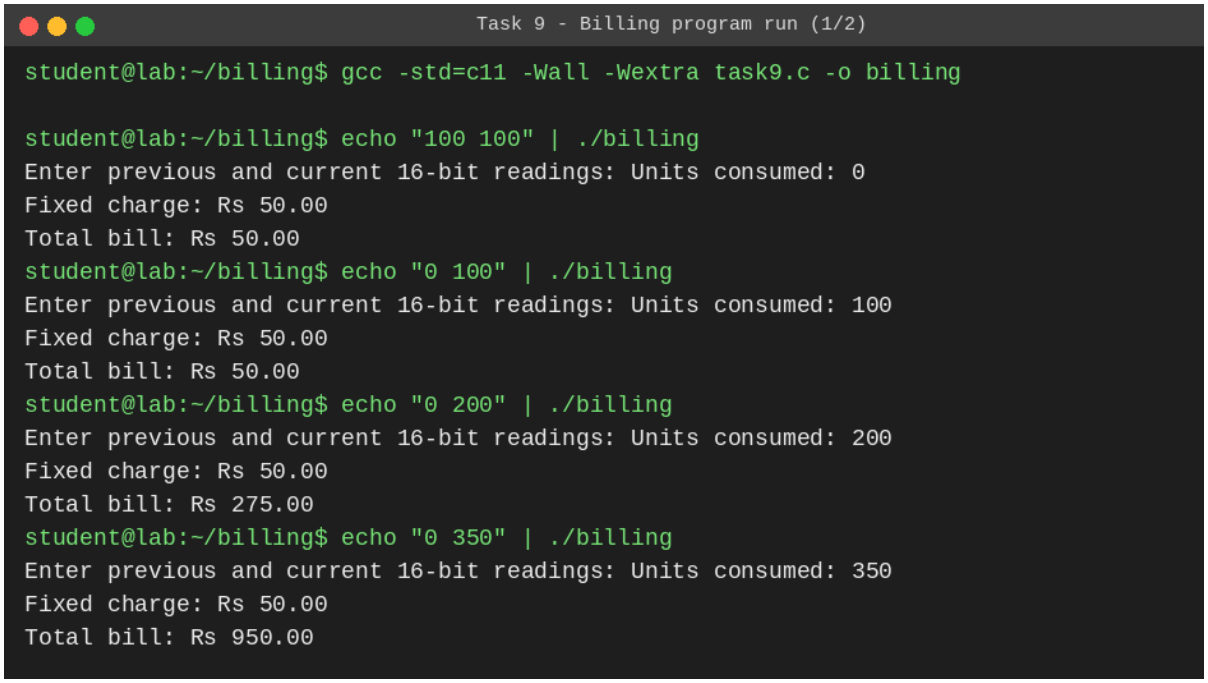


Figure 5: Billing program – zero, 100, 200 and 350 units

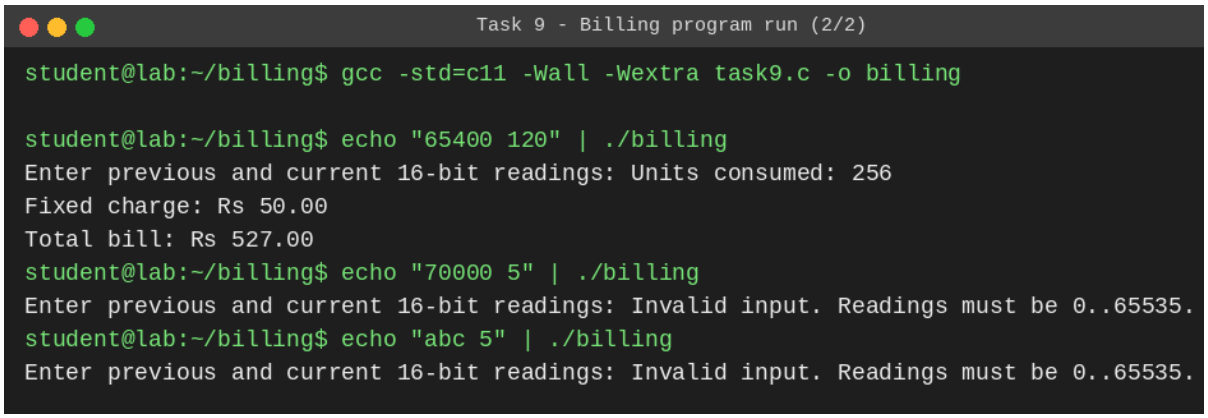


Figure 6: Billing program – rollover, invalid range and bad text

Test table (expected vs actual)

Test	Input / units	Expected	Actual	Result
Zero usage	previous=100, current=100; 0 units	Rs 50.00	Rs 50.00	Pass
Exactly 100	0 to 100; 100 units	Rs 50.00	Rs 50.00	Pass
Exactly 200	0 to 200; 200 units	Rs 275.00	Rs 275.00	Pass
Above 200	0 to 350; 350 units	Rs 950.00	Rs 950.00	Pass
Rollover	65400 to 120; 256 units	Rs 527.00	Rs 527.00	Pass
Invalid range	previous=70000	Reject input	Reject input	Pass
Bad text	previous=abc	Reject input	Reject input	Pass

Rollover check: Rs 50 + 100 × 2.25 + 56 × 4.50 = Rs 527.00. Observation: when current is lower than previous (for example 500 then 100), this program treats it as a rollover and bills 65136 units. A real system should treat such a case as a possible meter fault or reset and flag it, which is why the report recommends timestamp and plausibility validation.

TASK 10 – Compare Expectations with Execution (4 marks)

For a typical tariff bill, total paise = 5000 + max(min(units−100,100),0)×225 + max(units−200,0)×450. At 4,000 households, 60,000 households and 800,000 households, total collection is the sum of the individual household bills; it cannot be known from household count alone without each household's consumption.

An int may overflow if a district's total paise is accumulated into a 32-bit signed value. long may be 32-bit on some Windows compilers, so long long or uint64_t is safer. For example, 800,000 bills averaging Rs 950 yield Rs 760,000,000, or 76,000,000,000 paise; the paise total exceeds a signed 32-bit integer. Floating-point totals may differ by tiny fractions before rounding; integer paise provides exact totals as long as the chosen integer type has sufficient range.

A double district total generally retains much more range than int, but floating-point precision is finite and not every integer above 2^53 is represented exactly. For currency, use integer paise and check addition/multiplication bounds.

TASK 11 – Recommend and Defend the Final Design (4 marks)

- Use unsigned fixed-width or validated integer types for meter readings, and uint64_t / int64_t for cumulative quantities and money where supported.
- Represent all currency as integer paise; format rupees and paise only when displaying the bill.
- Keep tariff values in one central configuration. For the original exercise, macros are acceptable; for real tariff revisions, load a validated configuration file at start-up or use an administrative database.
- Validate each reading, timestamp, tariff and intermediate arithmetic. Log changes with effective dates and preserve an audit trail.
- At district scale, use 64-bit checked totals and test maximum expected consumption, not just typical values.

- Limitations: the simple example handles one 16-bit rollover only and does not implement taxes, arrears, missing readings, tamper detection or multiple tariff categories.

A configuration file adds O(k) start-up time and O(k) storage for k tariff entries, but tariffs can change without recompiling and redistributing the executable. Compiling constants into the program has near-zero tariff-loading cost but each revision requires a rebuild and deployment.

TASK 12 – Re-evaluate and Adapt the Design (8 marks)

A. Impact analysis

The new meter sends watt-hour readings every 15 minutes, producing 35,040 readings per year per household. A daily time-of-day tariff applies, and usage between 6 PM and 10 PM costs 20% more. Tariff values must be changeable without recompilation. A single cumulative reading difference no longer tells how much energy was used during peak hours; interval readings and timestamps are required. Annual watt-hour totals and district aggregates need wider integer types.

B. Evaluate the original design

The original unsigned short is sufficient only for a single 16-bit register value, not for annual energy totals. At 35,040 intervals, the system should store cumulative watt-hours in uint64_t or use validated interval deltas. #define constants cannot be changed at runtime, so tariff values must move to a configuration structure or file. The original fixed-slab function remains useful for non-time-dependent slabs but must be extended to apply a peak multiplier.

C. Adapted design and example code

```
#include <stdio.h>
#include <stdint.h>
typedef struct {
 int64_t fixed_paise;
 int64_t rate2_paise;
 int64_t rate3_paise;
 int peak_start_hour; /* inclusive: 18 */
 int peak_end_hour; /* exclusive: 22 */
 int peak_percent; /* 120 means 120% of base rate */
} Tariff;
int is_peak_hour(int hour, const Tariff *t)
{
 return hour >= t->peak_start_hour && hour < t->peak_end_hour;
}
int64_t interval_charge_paise(uint64_t watt_hours, int hour,
 int64_t rate_paise_per_kwh,
 const Tariff *t)
{
 uint64_t numerator = watt_hours * (uint64_t)rate_paise_per_kwh;
 if (is_peak_hour(hour, t))
 numerator = numerator * (uint64_t)t->peak_percent / 100U;
 return (int64_t)((numerator + 999U) / 1000U); /* round up to paise */
}
int main(void)
{
 Tariff t = {5000, 225, 450, 18, 22, 120};
 printf("Peak 19:00? %s\n", is_peak_hour(19, &t) ? "yes" : "no");
 printf("Peak 22:00? %s\n", is_peak_hour(22, &t) ? "yes" : "no");
 printf("250 Wh at 19:00, Rs 2.25/kWh slab -> %lld paise\n",
 (long long)interval_charge_paise(250, 19, 225, &t));
 printf("250 Wh at 10:00, Rs 2.25/kWh slab -> %lld paise\n",
```

```
 (long long)interval_charge_paise(250, 10, 225, &t));
 return 0;
}
```

```
Task 12 - Adapted design output

student@lab:~/billing$ gcc -std=c11 -Wall -Wextra task12.c -o task12

student@lab:~/billing$ ./task12
Peak 19:00? yes
Peak 22:00? no
250 Wh at 19:00, Rs 2.25/kWh slab -> 68 paise
250 Wh at 10:00, Rs 2.25/kWh slab -> 57 paise
```

Figure 7: Output of the adapted smart-meter design

The example demonstrates the peak-hour rule and runtime tariff structure. As a worked check, 250 Wh at the 225 paise/kWh rate costs 56.25 paise off-peak, rounded up to 57; at 120% (peak) it is 67.5, rounded up to 68 – matching the program output. For a complete deployment, parse a configuration file into Tariff, validate non-negative rates and valid hour ranges, and reject the file if any field is missing or outside limits. The interval calculation should use checked multiplication to prevent overflow; for exact kWh pricing, retain the numerator and apply the approved rounding rule once at the invoice level.

D. Evidence-based decision

Decision: MODIFY the original design. Keep integer paise and progressive slab calculations, but replace the single-reading model with timestamped interval records, widen cumulative types, and move tariff parameters to a validated runtime configuration. At 35,040 records per household annually, 4,000 households produce 140,160,000 records/year; 60,000 produce 2,102,400,000; 800,000 produce 28,032,000,000. This scale requires database storage, batching, indexing, retention rules and careful capacity planning. A single-process demonstration is not sufficient for district-scale production.

Build and verification checklist

- Compile the original examples with warnings enabled and record the actual compiler/version and output (GCC 13.3.0 used here; no warnings).
- Test zero, 100, 200 and 350 units, rollover, invalid text, negative values, and current reading lower than previous without a rollover event.
- Verify all bill totals in integer paise and compare expected with actual results.
- Inspect macro expansion using gcc -E and compilation stages using gcc -S and gcc -c.
- Commit each task progressively to a public or course-approved private GitHub repository named CSA02-1912260571-Assignment1.
- Keep one folder per task with source, input/output evidence and a short README.

Conclusion

The recommended design uses validated integer inputs, unsigned arithmetic for the original 16-bit meter rollover, integer paise for money, centrally managed tariff parameters, correct format specifiers, and tests at slab boundaries. The smart-meter requirement needs a modified design with timestamped watt-hour intervals, a peak-hour multiplier, wider counters and runtime tariff configuration. These choices reduce rounding errors, prevent common overflow mistakes and make tariff revisions easier to audit.

References / Notes

- C language standard library headers: stdio.h, limits.h, stdint.h.
- GCC compiler options used in this report: -E, -S, -c, -Wall, -Wextra and -DDEBUG.
- Expected results are calculated from the tariff stated in the assignment and confirmed by running the programs (GCC 13.3.0).